

```
# Ayush Shah C080
```

```
# Aim - TO Identify the edges on an image using the sobel filter. Observe the changes wen the filser size increases
```

```
# Task - Identify edges in the given image using sobel filter of size 3x3
```

```
# Increase the size of sobel filter and observe the effect on the filter unit
```

```
# Apply the same algorithm on any other image
```

```
import cv2
import numpy as np
from skimage import data
import matplotlib.pyplot as plt
```

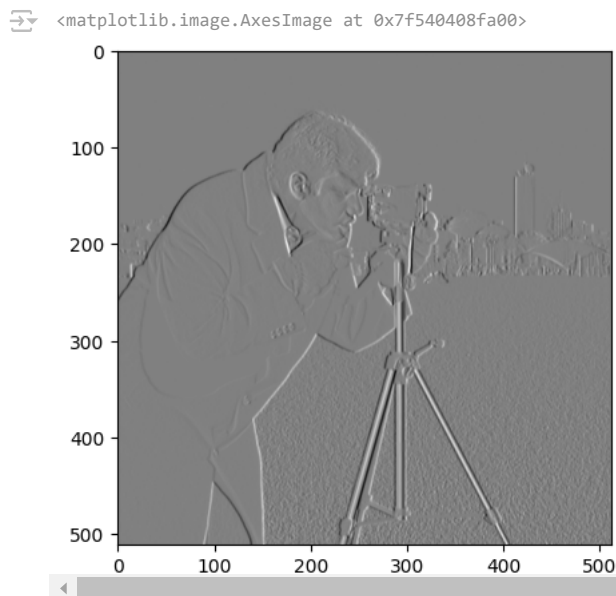
```
image = data.camera()
plt.imshow(image, cmap='gray')
```



```
image.shape
[rw, col] = image.shape
```

```
grad_x = cv2.Sobel(image, ddepth=cv2.CV_32F, dx=1, dy=0, ksize=3)
grad_y = cv2.Sobel(image, ddepth=cv2.CV_32F, dx=0, dy=1, ksize=3)
```

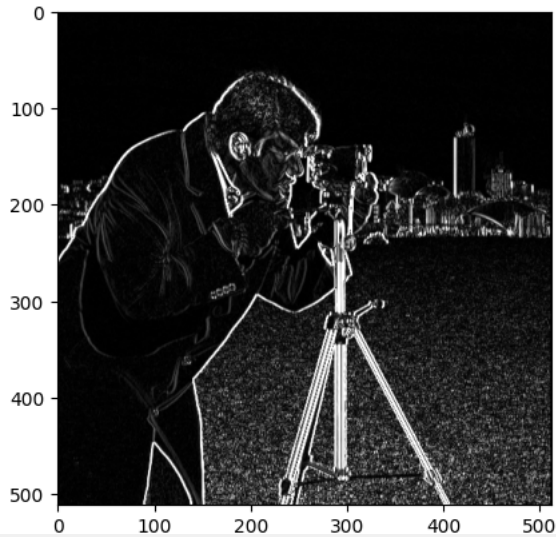
```
plt.imshow(grad_x, cmap='gray')
```



```
grad_x_abs = cv2.convertScaleAbs(grad_x)
grad_y_abs = cv2.convertScaleAbs(grad_y)
```

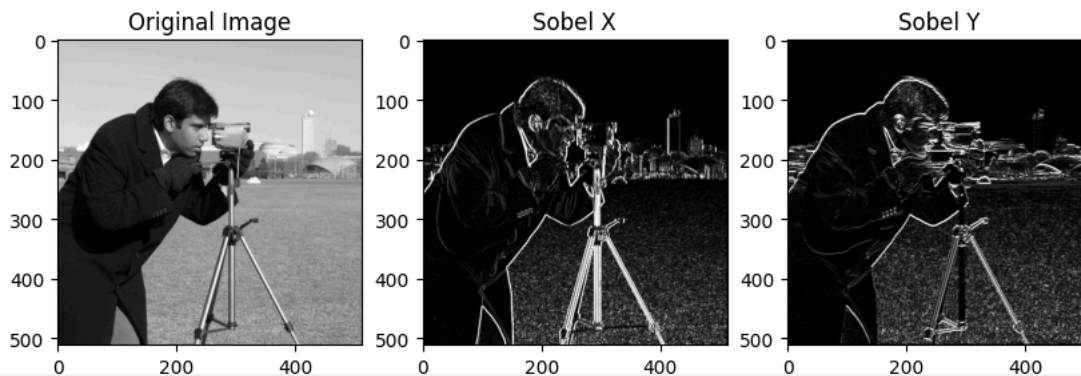
```
plt.imshow(grad_x_abs, cmap='gray')
```

 <matplotlib.image.AxesImage at 0x7f540e1d1660>



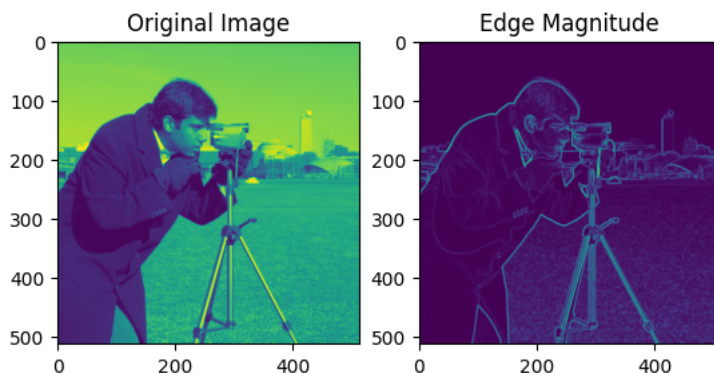
```
plt.figure(figsize=(10,5))
plt.subplot(1,3,1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.subplot(1,3,2)
plt.imshow(grad_x_abs, cmap='gray')
plt.title('Sobel X')
plt.subplot(1,3,3)
plt.imshow(grad_y_abs, cmap='gray')
plt.title('Sobel Y')
```

 Text(0.5, 1.0, 'Sobel Y')



```
edge_mag = np.sqrt((grad_x**2) + (grad_y**2))
plt.subplot(1,2,1)
plt.imshow(image)
plt.title('Original Image')
plt.subplot(1,2,2)
plt.imshow(edge_mag)
plt.title('Edge Magnitude')
```

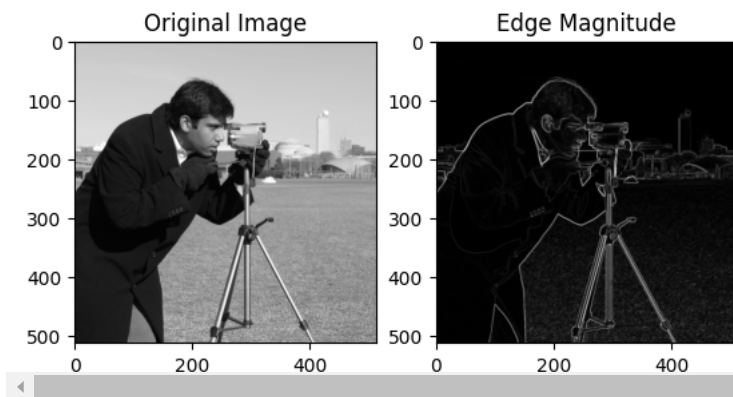
 Text(0.5, 1.0, 'Edge Magnitude')



```
edge_mag = np.sqrt(np.square(grad_x) + np.square(grad_y))
plt.subplot(1,2,1)
```

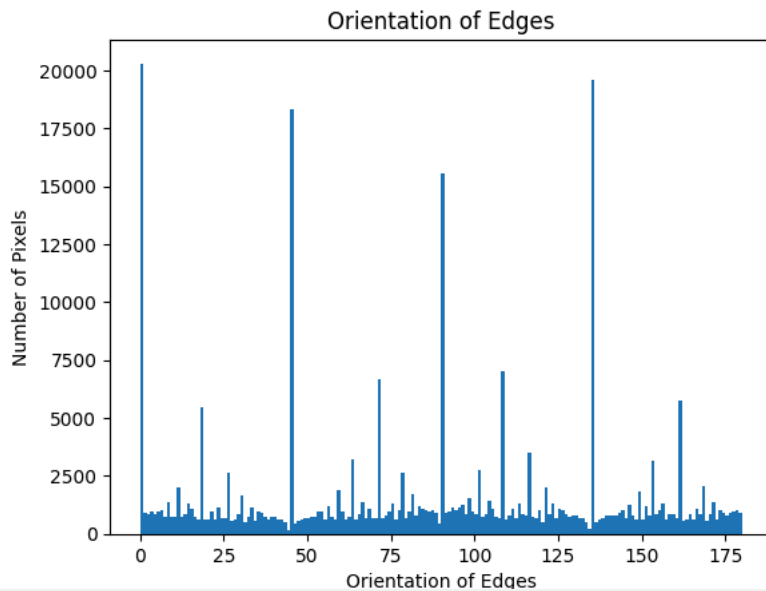
```
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.subplot(1,2,2)
plt.imshow(edge_mag, cmap='gray')
plt.title('Edge Magnitude')
```

 Text(0.5, 1.0, 'Edge Magnitude')



```
angle=np.arctan2(grad_y,grad_x)*180/np.pi%180
angleid=np.reshape(angle,(rw*col,1))
plt.hist(angleid,bins=180)
plt.xlabel('Orientation of Edges')
plt.ylabel('Number of Pixels')
plt.title('Orientation of Edges')
plt.show()
```





```
# Conclusion:
# 3x3 Sobel filter detects small edges of the given image.
# About 2000 pixels have edges with angle 0 degree.
# About 17500 pixels have edges with angle 135 degree.
# Maximum number of pixels have angles approximately 0, 45, 85, 135 degrees.
```

```
import cv2
import numpy as np
from skimage import data
import matplotlib.pyplot as plt
```

```
image = "/content/images.jpg"
image = cv2.imread(image)
image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
plt.imshow(image, cmap='gray')
```

 <matplotlib.image.AxesImage at 0x7f53f57ada50>

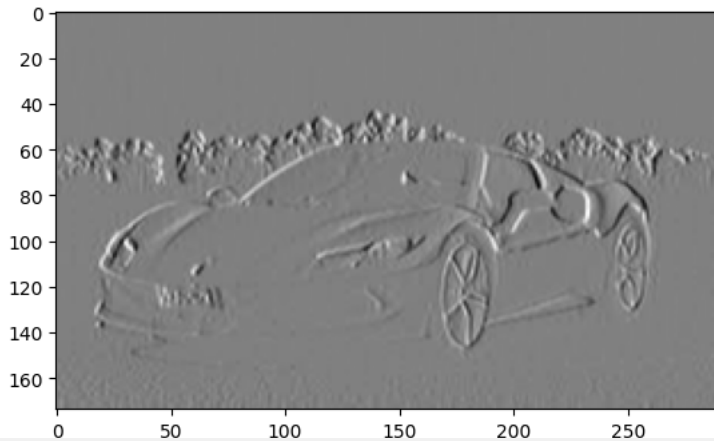


```
image.shape
[rw, col] = image.shape
```

```
grad_x = cv2.Sobel(image, ddepth=cv2.CV_32F, dx=1, dy=0, ksize=3)
grad_y = cv2.Sobel(image, ddepth=cv2.CV_32F, dx=0, dy=1, ksize=3)
```

```
plt.imshow(grad_x, cmap='gray')
```

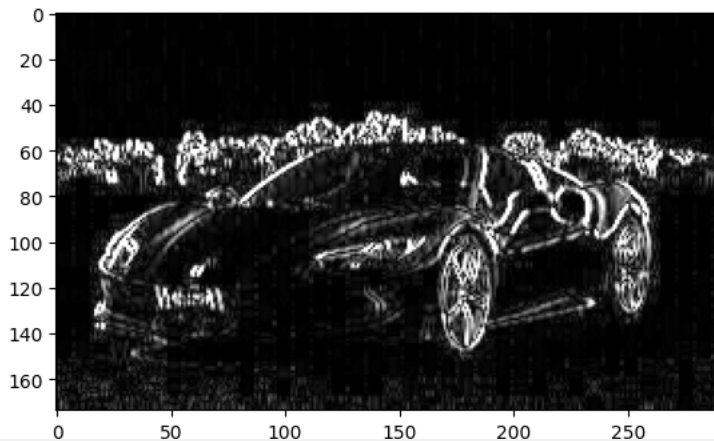
 <matplotlib.image.AxesImage at 0x7f53f51bb8b0>



```
grad_x_abs = cv2.convertScaleAbs(grad_x)
grad_y_abs = cv2.convertScaleAbs(grad_y)
```

```
plt.imshow(grad_x_abs, cmap='gray')
```

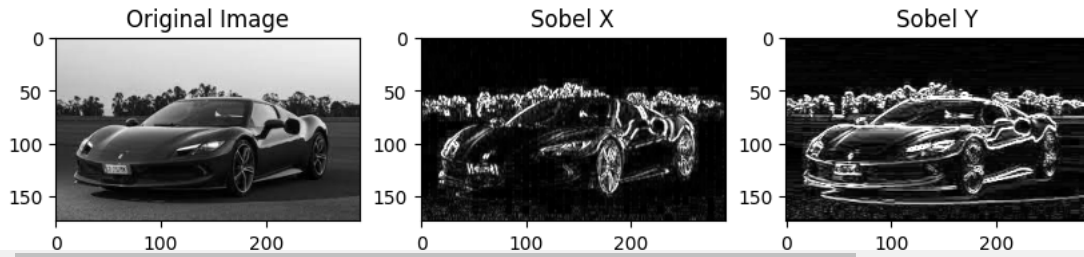
 <matplotlib.image.AxesImage at 0x7f53f5062470>



```
plt.figure(figsize=(10,5))
plt.subplot(1,3,1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.subplot(1,3,2)
plt.imshow(grad_x_abs, cmap='gray')
plt.title('Sobel X')
```

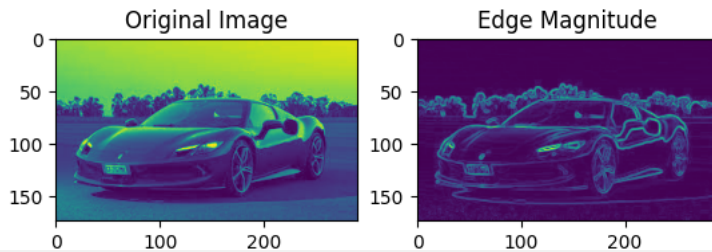
```
plt.figure(figsize=(10,10))
plt.subplot(1,3,3)
plt.imshow(grad_y_abs, cmap='gray')
plt.title('Sobel Y')
```

↻ Text(0.5, 1.0, 'Sobel Y')



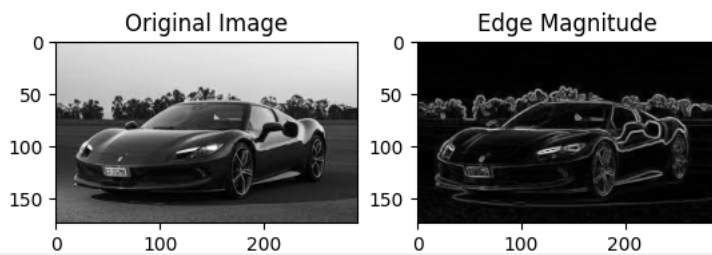
```
edge_mag = np.sqrt((grad_x**2) + (grad_y**2))
plt.subplot(1,2,1)
plt.imshow(image)
plt.title('Original Image')
plt.subplot(1,2,2)
plt.imshow(edge_mag)
plt.title('Edge Magnitude')
```

↻ Text(0.5, 1.0, 'Edge Magnitude')



```
edge_mag = np.sqrt(np.square(grad_x) + np.square(grad_y))
plt.subplot(1,2,1)
plt.imshow(image, cmap='gray')
plt.title('Original Image')
plt.subplot(1,2,2)
plt.imshow(edge_mag, cmap='gray')
plt.title('Edge Magnitude')
```

↻ Text(0.5, 1.0, 'Edge Magnitude')



```
angle=np.arctan2(grad_y,grad_x)*180/np.pi%180
angleid=np.reshape(angle,(rw*col,1))
plt.hist(angleid,bins=180)
plt.xlabel('Orientation of Edges')
plt.ylabel('Number of Pixels')
plt.title('Orientation of Edges')
plt.show()
```

